Analysis of Algoritms

Assignament 06

Name: Jose Ignacio Cabascango Fernandez

Link GitHub: https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignament-06/src

1. Solve the following recurrences using the Master Theorem:

1. Solve the following recurrences using the Master Theorem.

1.1 $T(n) = 4T(n/2) + n, \quad T(1) = 1$

$a = 4, \quad b = 2, \quad d = 1$

who, $a > b$

$4 > 2^1$

use

$T(n) = n^{\log_b a} = n^{\log_2 (4)} = n^2$

$T(n) \in O(n^2)$

- Conclusion: The predominant cost stems from recursive calls.

1.2 $T(n) = 4T(n/2) + n^2, \quad T(1) = 1$

$a = 4, \quad b = 2, \quad d = 2$

who $a = b^d$

$4 = 2^2$

use

$T(n) = n^d \log n = n^2 \log n$

$T(n) \in O(n^2 \log n)$

- Conclusion: The work at each level of recursion accumulates over approximately $\log_2 n$ levels.

1.3 $T(n) = 4T(n/2) + n^3, \quad T(1) = 1$

$a = 4, \quad b = 2, \quad d = 3$

who $a < b^d$

$4 < 2^3$

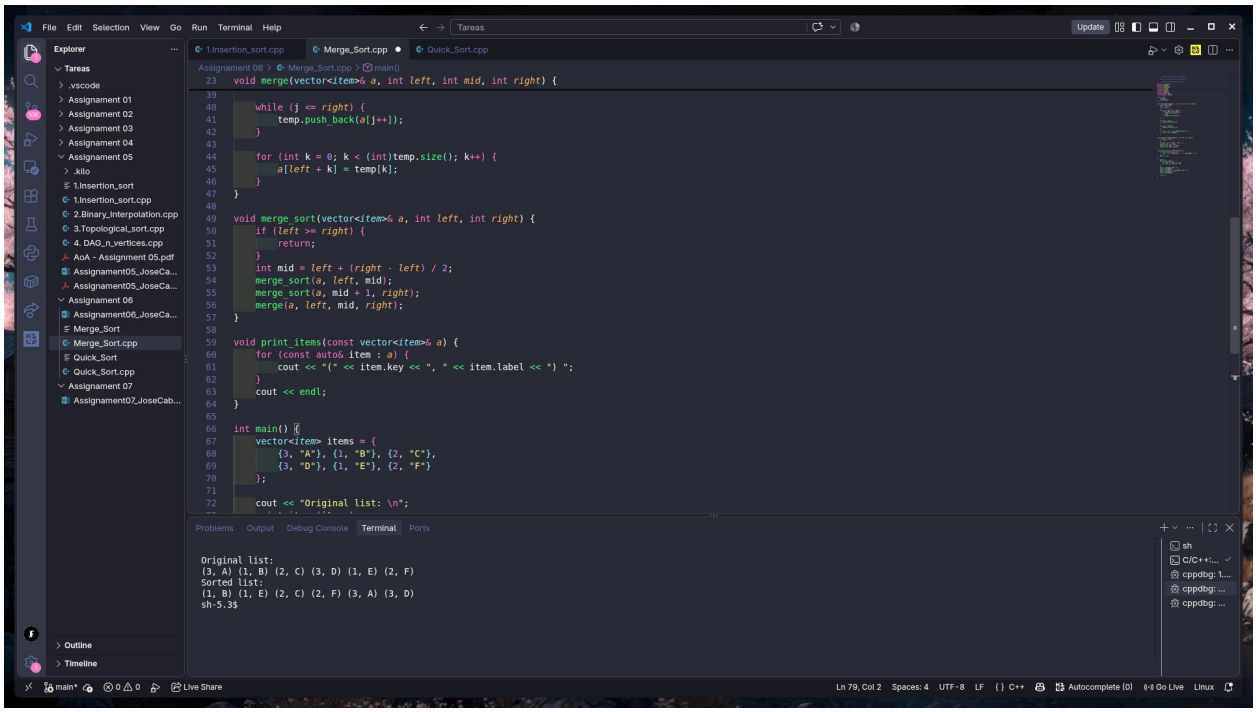
use

$T(n) = n^d = n^3$

$T(n) \in O(n^3)$

- Conclusion: The additional cost n^3 dominates the growth of recurrence.

2. Implement the Merge Sort algorithm.



3. Use your implementations and experiment to check if the merge sort is or is not a stable sorting algorithm. Provide and example.

A sorting algorithm is stable if it preserves the relative order of elements that have the same key.

Consider the original list:

Input: (3, A) (1, B) (2, C) (3, D) (1, E) (2, F)

Expected result: (1, B) (1, E) (2, C) (2, F) (3, A) (3, D)

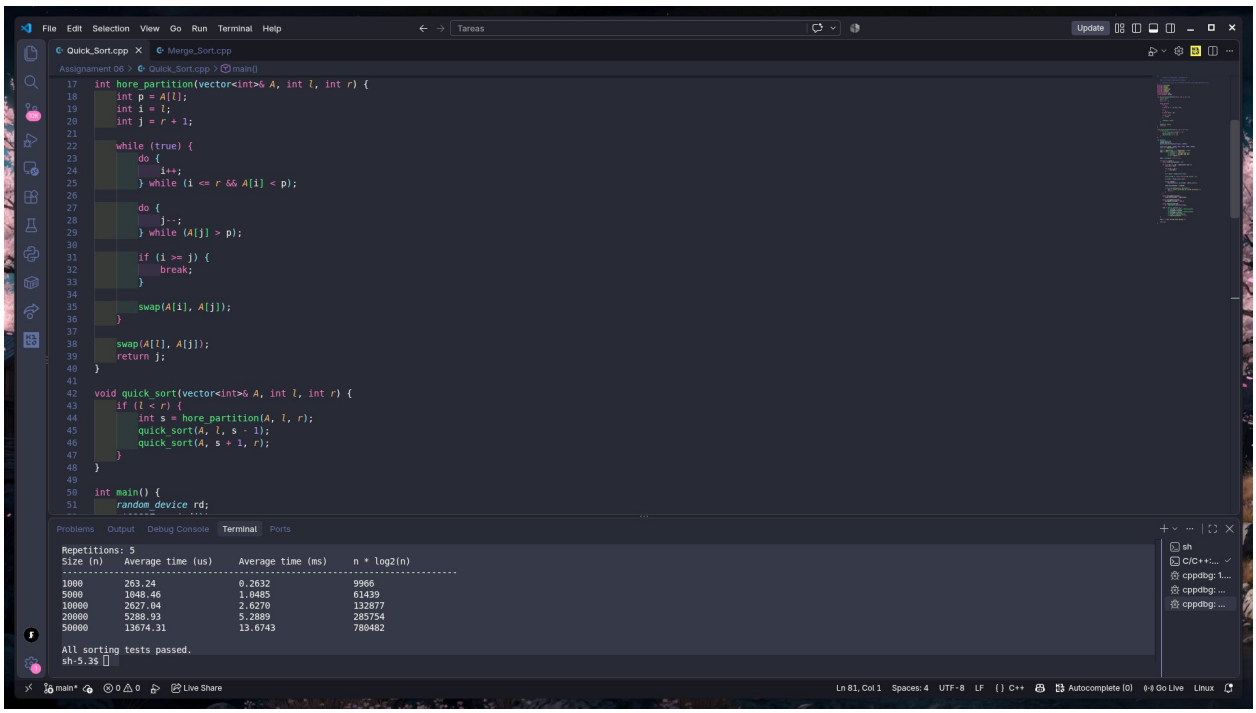
Note that:

- For key 1, the order B, E is preserved.
- For key 2, the order C, F is preserved.
- For key 3, the order A, D is preserved.

This implementation of Merge Sort is stable. The key lies in using <= when comparing equal elements, which gives priority to the element that appears first in the left half.

Stability depends on the implementation; the merge operation must preserve the relative order.

4. Implement the Quick Sort algorithm and test using random generated lists.



5. Apply empirical analysis on the quick sort. Does it match with the mathematical analysis?

Experimental results

The Quick Sort algorithm was tested using randomly generated lists of different sizes. For each input size, the program performed 5 repetitions and calculated the average execution time.

Input size (n)	Average time (µs)	Average time (ms)	(nlog_2 n)
1,000	263.24	0.2632	9,966
5,000	1,048.46	1.0485	61,439
10,000	2,627.04	2.6270	132,877
20,000	5,288.93	5.2889	285,754
50,000	13,674.31	13.6743	780,482

All sorting tests passed, this confirm that all tested arrays were sorted correctly.

Analysis of the results

The execution time increases as the input size increases. For example, when the input size grows from 1,000 to 50,000 elements, the number of elements increases by a factor of 50, while the average execution time increases by approximately 51.95 times.

This increase is broadly consistent with the expected average-case behavior of Quick Sort, O(n log n). However, the execution time does not have to grow at exactly the same rate as n log n, because actual performance depends on pivot selection, the distribution of the input values, and hardware conditions.

Comparing theoretical and empirical growth

For the average case, Quick Sort follows the recurrence

$$T(n)=2\,T(n/2) + n,$$

when partitions are approximately balanced. By the Master Theorem, this gives

$$T(n)=O(n \log n)$$

To compare the measurements more directly, consider the ratio between the execution time and (n log_2 n):

(n)	Time / (n log_2 n) (µs per unit)
1,000	0.0264
5,000	0.0171
10,000	0.0198
20,000	0.0185
50,000	0.0175

The ratio becomes relatively stable for the larger inputs. This supports the interpretation that the observed behavior is consistent with (n log n) growth, although these measurements alone do not prove the asymptotic complexity.

As the input size increases, the execution time grows at a rate that is broadly compatible with the theoretical model.